

Module 3 Narrative

William Mitchem

Bachelor of Science in Computer Science, SNHU

CS 499: Computer Science Capstone

Dr. Satish Penmatsa

May 24th, 2026

Module 3 Narrative

The artifact I chose was the full-stack SPA that I created as part of CS 465: Full Stack Development in the term before this one. The project consisted of a legacy Angular frontend, Node/Express backend, and a MongoDB database. Its purpose was to market trips offered by the Travlr travel agency.

Artifact Justification

I chose this artifact because it's expansive enough to offer opportunities for meeting all three enhancements, while also affording me the opportunity to shift the purpose of the software into what is, for me, a passion project. It's also the kind of work I want to do after graduation. That's why my concentration is in Software Engineering as well.

How the Artifact Was Improved

I migrated the codebase from Angular to React, implementing a strict feature-driven architecture, rather than the flat, disorganized original tree. I chose that approach so that I could inherently isolate the features related to the original purpose of the app (trips), to reduce friction as I add the new features.

From a basic functional perspective, the Angular version lacks a registration page entirely. Creating accounts was handled exclusively by Postman. With the React version, that's been rectified with a new registration page. Another gripe with the original artifact is that selecting a trip to edit does not populate the fields of that trip automatically, which drastically impedes the ability to update individual fields as you previously needed to memorize what all the other fields were and retype them just to change one of the fields.

To handle state management across this new feature-driven architecture, I had to intentionally separate my global, server, and local states. For application-wide data like user

authentication and UI themes, I utilized React's Context API so any component could access that data without messy prop-drilling. For managing server data, like the fetched Pokémon lists, I used TanStack Query to cache responses and prevent redundant backend API calls. Finally, for localized component behavior, like opening a mobile menu or tracking search bar inputs, I relied on standard React hooks like useState. This separation kept my application's memory footprint light and organized.

Throughout the React migration and TypeScript integration, I also had to adapt my testing and validation strategies. Moving away from Angular's highly opinionated environment meant I had to build my own guardrails. I relied heavily on TypeScript for static validation; by strictly typing my component props and API payloads, the compiler acted as an automated testing layer, catching mismatch errors and missing variables before I even ran the code in the browser. When I did encounter runtime issues or infinite fetch loops, I utilized the React Developer Tools browser extension to inspect the virtual DOM and trace exactly how state was flowing through my 'smart' containers, which made debugging significantly faster than relying on traditional console logs.

Showcasing Skills

This artifact highlights my ability to rearchitect a frontend infrastructure without breaking the existing backend API contract. It also showcases my proficiency with modern component-based frameworks (React 19), strict type-safety (TypeScript), and advanced UI/UX responsive layouts.

Did I Meet My Planned Course Outcomes?

Yes, I met my planned course outcomes and satisfied the requirements for the Software Design & Engineering outcome. My goal for the enhancement was to achieve feature parity with

the Angular version, which I achieved, while adding some features that the original artifact should've always had.

My overall course plan is still on track, and I successfully laid down a much stronger architecture foundation than I initially planned by implementing feature-driven architecture. Since the UI is highly modular, it should be easy for me to shift the goal of the software with the next two enhancements.

Reflection

A major source of my learning for this first enhancement relates to my coding history. Most of my coding history for personal projects has been OOP-based; C#, C++, Java, Python. Angular is a natural extension of that class-based system but for web development. Conversely, React is very functional, which is a paradigm I don't engage with much. So, I chose the React migration explicitly because I wanted experience thinking about the architecture of my software in an entirely new way. Utilizing TypeScript rather than JavaScript helped with that process, because some of that OOP flavor is still present.

Coming from an OOP background, I have prior experience with separation of concerns, but I deepened my understanding of the practice. By isolating stateful views (like pages) from generic presentation elements (located in the components/ui folder), I learned how to make React components truly reusable.

Implementing all the different parts of the stack was definitely challenging. Angular is much more monolithic than React, it's very opinionated and there's only one way to approach most features. I've appreciated the agency that React provides in being a more pared down framework, allowing me to piece together my own stack, but there was a steep learning curve ensuring every piece of the pipeline played nicely.